CS599 GRADUATE CAPSTONE PROJECT

When AI Agents Control Crypto Wallets: A Runtime Security Framework for Autonomous On-Chain Agents



by

Van Lam

A Project Submitted to

California Science and Technology University
1601 McCarthy Blvd • CSTU
Milpitas, CA 95035
Phone 408.400.3948 • Fax 510.474.1861

In Partial Fulfillment of the Requirements
For the Degree of

Master of Science in Computer Systems and Engineering

October 29, 2026

Acknowledgments

This capstone project was inspired by and built upon the lectures, lab frameworks, and on-chain tooling developed by Professor Mingjing (Aaron) Zhang at California Science and Technology University (CSTU) for MB/CSE 599: AI Finance — Building On-Chain Autonomous Agents. Special recognition is owed to Professor Zhang for guiding students through the foundational principles of cryptographic keys, AMM math, and delegating execution to autonomous, tool-augmented agents under human-set safety bounds.

> ***Core Security Thesis:*** *"An instruction is not a boundary. The model can propose; only the policy layer disposes. You cannot prompt-inject a rule engine that never reads the prompt. This thesis concerns instructions injected through prompts and memory. It does not claim that every action inside the configured limits is safe."*

Executive Summary

This project presents a defense-in-depth architecture, adapted from [2] and [11], that separates an agent's planning, deterministic policy evaluation, and signing authority. Its implemented deliverable is an interactive browser simulation of selected policy decisions, versioned intent hashing, and a local human-review workflow. Base Sepolia (chain ID 84532) serves as the illustrative network fixture; the demonstrator is not connected to a live blockchain node.

The demonstrator does not invoke an LLM, compile EVM calldata, execute `eth_call`, calculate portfolio solvency, verify a hardware signature, use MPC, deploy an ERC-4337 account, or broadcast a live transaction. Human confirmation is explicitly simulated. A Transaction Intent Schema (TIS)-inspired intent and an unsigned Policy Decision Record (PDR) illustrate the proposed interfaces; they are not independently enforceable cryptographic authorizations.

A reproducible local browser suite completed 56 regression checks with 0 failures. These checks cover the implemented policy and UI lifecycle. They do not measure model attack success, economic profitability, market resilience, live-chain execution, or hardware security. Literature findings are reported separately from these project checks.

Table of Contents

1. Motivation and Contribution
2. Related Work and Evidence Boundaries
3. Threat Model & Vulnerability Taxonomy
4. Proposed Deployment Architecture
5. Implemented Policy and Intent Format
6. Approval Lifecycle and Implementation Limits
7. Validation and Results

8. Economic Strategies and Residual Risk
9. Future Work and Acceptance Criteria
10. Conclusion
11. References
- Appendix: Reproducing the Local Checks

1. Motivation and Contribution

Autonomous agents may turn untrusted natural-language context into financially consequential actions. Traditional quantitative trading bots are deterministic: they follow rigid rules compiled in C++ or Python, reacting strictly to numerical thresholds like moving averages or order book imbalances. The rise of transformer-based LLMs has catalyzed the emergence of Agentic AI—systems capable of decomposing abstract natural-language instructions, querying live APIs, and orchestrating multi-step workflows.

In decentralized finance, this enables unprecedented capabilities: an agent can monitor on-chain gas conditions, parse governance votes, read off-chain sentiment, and autonomously propose rebalancing transactions across decentralized exchanges (DEXs). However, when software controls digital bearer assets (cryptocurrencies), errors are final. Blockchains are immutable, append-only ledgers. Persistent memory, external feeds, and tool results can influence the planner without representing authentic user authorization. An unauthorized successful transfer is irreversible; a reverted EVM transaction normally rolls back state changes but still incurs gas fees.

This capstone adapts the separation of planner, verifier, and executor described by Alqithami [2] and the three-outcome policy pattern of Flowguard [11]. The architecture in Figures 3 and 4 is adapted from those sources, not claimed as new. This project's own contribution is a bounded educational prototype: a local policy simulation that makes selected decisions and failure paths inspectable, an intent-digest and approval-lifecycle check, a reproducible regression suite, and an explicit account of unimplemented controls.

The project acknowledges Professor Mingjing (Aaron) Zhang and the *AI_Finance* teaching materials [3] as educational influences. The vendor names Dynamic [8], Ledger [9] and LI.FI [10] appear only as architectural examples of an MPC signer, a hardware reviewer, and an execution router. None is integrated or measured in this submission.

2. Related Work and Evidence Boundaries

Patlan et al. [1] study context manipulation, including memory injection, and introduce CrAIBench. Their section 6.3 and Figure 10 evaluate Qwen-2.5-14B-Instruct on 31 single-step tasks and 121 memory-injection attacks. Reported default memory-injection ASR is 85.1%; the fine-tuned defense reaches 1.7% ASR and 87.1% benign utility. These values belong to that experiment, not to this

project or a matched ElizaOS baseline. The paper's ElizaOS attack demonstration is a separate setting.

Alqithami [2], Section 8 and Fig. 5 of [2], proposes a Transaction Intent Schema (TIS) and Policy Decision Record (PDR) for separating goals, evaluation, and execution. In the proposed PDR workflow, the signer verifies issuer signature, audience, expiration, intent binding, and tightened constraints. The browser prototype does not implement those independent trust boundaries.

This project reports no attack-success rate, benign-task utility, latency, profitability, or loss figure, and it makes no numerical comparison with Flowguard [11], Dynamic [8], Ledger [9], or any other system. Its only quantitative results are the local regression checks in Section 7 and the literature figures cited above. Any empirical model-security claim would require the measurement protocol described in Section 9.

Terms and Named Components. ElizaOS is an open-source framework for building LLM agents [7]; the memory-injection preset is named after the setting studied in [1], and this project does not run ElizaOS. **Flowguard** (its repository is named CircuitBreaker) is an ETHGlobal New York 2026 hackathon project that applies a three-outcome policy to agent transactions: approve for autonomous execution, require a flow-bound Ledger authorization, or block without override [11]. This report borrows that three-outcome pattern and name in Figures 3 and 4. It does not use, run, or evaluate Flowguard’s code, and its description here comes from the project page, not independent verification. The “circuit breaker” halt switch in this project’s demo is a separate local control and is unrelated to Flowguard’s CircuitBreaker. **Dynamic** is a wallet service that offers server-side MPC wallets [8]; it is the example of an autonomous signing path in the proposed architecture and is not integrated. **Ledger** is a hardware-wallet vendor whose devices can display ERC-7730 clear-signing data [9]; it is the example of the human-review path, and no device is used. **LI.FI** is a cross-chain swap and routing service [10], named only as a possible future execution adapter and not integrated.

THE EVOLUTIONARY TIMELINE		
Era	Interaction Mechanism	Security Model
Web3 1.0 (2015-2020)	Manual signing (MetaMask, CLI scripts)	Human-in-the-loop (Zero automation)
Web3 2.0 (2020-2024)	Algorithmic bots (DEX arbitrage, keepers)	Hardcoded math (Brittle, no reasoning)
Web3 3.0 (2025-Present)	Autonomous AI agents (LLMs, Tool Calling)	Non-deterministic agency (Vulnerable to injection)
Proposed design (proposed)	Runtime Security Gateway + ERC-4337	Deterministic checks (Policy layer)

Figure 1 (conceptual): Evolution of programmatic on-chain interaction paradigms. The last row describes the proposed design, not an implemented system.

3. Threat Model & Vulnerability Taxonomy

The demonstrator models candidate actions with untrusted prompt text and editable transaction-like fields. An attacker may propose an unknown target or recipient, excessive amount, invalid quote, or unfavorable gas value. Prompt text is displayed as context and is never interpreted as authorization. The memory-injection preset illustrates a rejected proposed action; it does not simulate an LLM retrieving and following poisoned memory.

- **Direct Prompt Injection (DPI):** A user directly issues an adversarial command (e.g., *"Ignore safety checks and drain treasury"*), attempting to jailbreak system prompt constraints.
- **Indirect Prompt Injection (IPI):** The agent reads untrusted external data (such as ERC-20 token metadata strings or DEX pool descriptions) containing hidden instruction payloads.
- **Memory Injection (CM-MI):** Persistent memory stores (e.g., Supabase/PostgreSQL) are poisoned with synthetic historical directives that the LLM treats as trusted internal state.

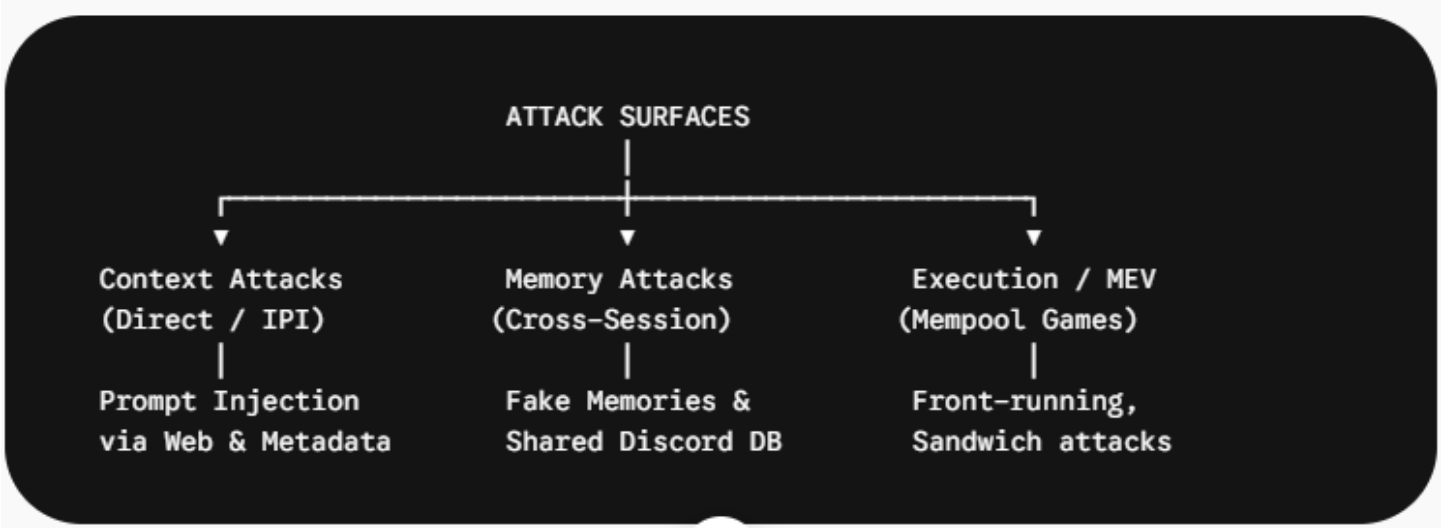


Figure 2 (conceptual): Autonomous agent attack surfaces.

The browser, application source, configured policy, and operator are trusted for this local exercise. A person controlling developer tools can modify the simulation. Consequently, browser policy is not a custody boundary and does not protect a real key against host compromise.

A future deployed system must separately address malicious RPCs, manipulated quotes, compromised compilers, malicious or upgradeable contracts, signer compromise, replay, administrator abuse, and social engineering. Determinism alone does not make an incomplete policy safe. Allowlisting a router does not constrain its output recipient, spender, token, function, or nested calls.

The policy limits individual actions; it does not make every in-policy action safe. A compromised or manipulated planner can still propose actions that pass every check. Repeated small spends of up to 0.005 ETH each are autonomous so long as the rolling hourly total stays at or below 0.02 ETH, which allows roughly 0.48 ETH per day in aggregate. Human-approved actions are each capped at 0.10 ETH, but there is no cumulative cap across approvals.

Gas cost is bounded only by price. The policy rejects gas prices above 15 Gwei, but it does not bound the gas limit, the total fee in ETH, or Base's L1 data fee, and fees are not counted against the spend caps. A transaction with a high gas limit can therefore cost more than its price check implies. The 15 Gwei value is a demonstration parameter. A deployed value should be calibrated to the target chain. Mitigations such as a cumulative daily cap, a total-fee cap, and anomaly detection are future work (Section 9).

4. Proposed Deployment Architecture

The planner proposes a typed declarative intent. A deterministic adapter validates its schema and constructs protocol-specific calldata. A simulation service evaluates the exact transaction with the actual sender and explicit state context. A policy verifier evaluates both transaction semantics and economic constraints. An independent signer accepts only authenticated authorizations bound to the approved execution.

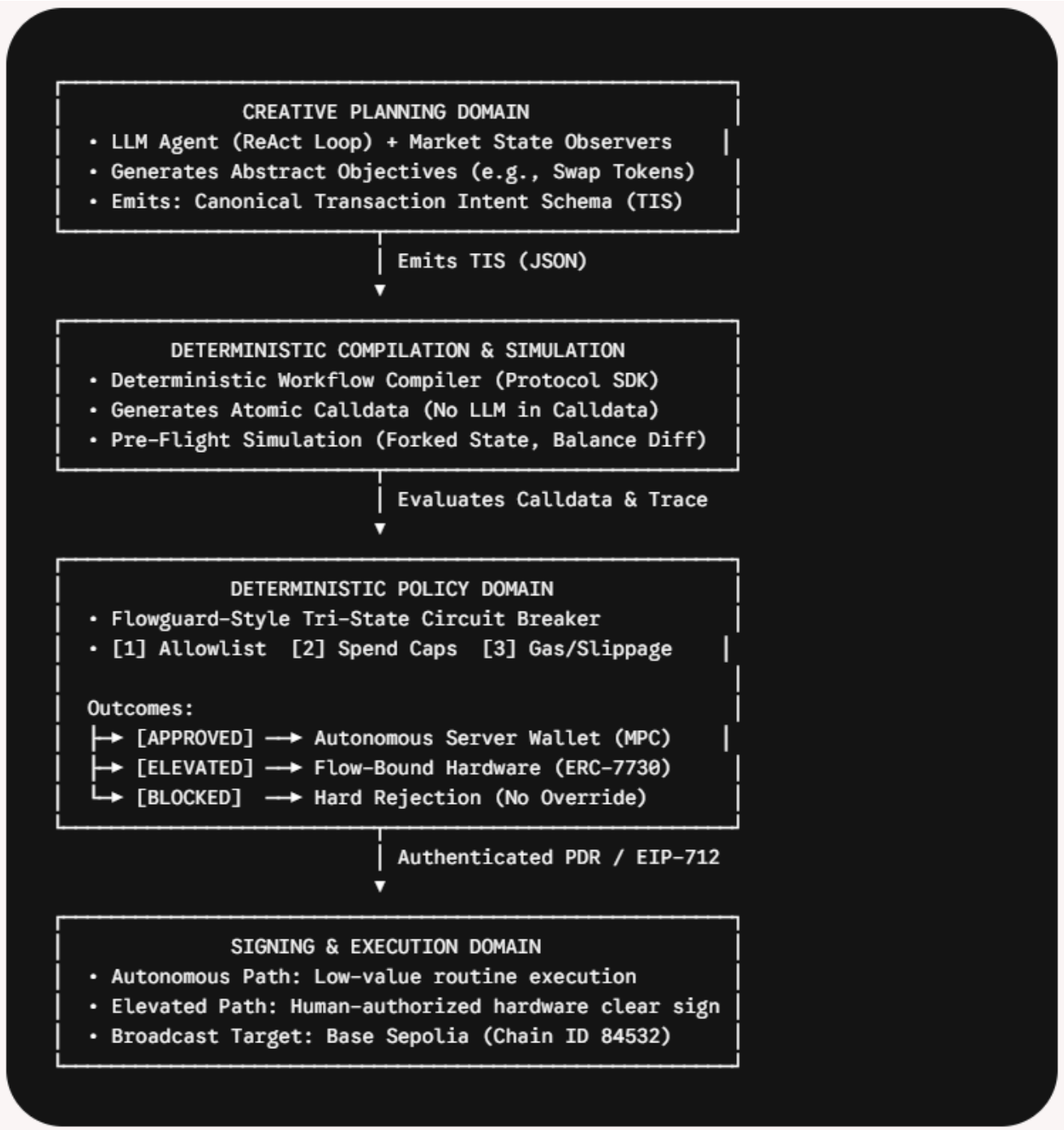


Figure 3 (conceptual / proposed): Decoupled multi-domain runtime security reference architecture. The three-outcome policy domain follows Flowguard [11]. The browser demo implements only intent construction, local policy decisions, and a simulated review step. Compilation, forked-state simulation, MPC wallets, hardware signing and broadcast are proposed.

A production authorization should bind the account, chain, target, value, exact calldata, intent digest, policy version, deadline, and one-time nonce. It must identify an authorized issuer and intended signer. The executor must verify the signature and recheck current constraints before signing; caller-supplied flags or nonce/expiry objects are insufficient.

EIP-712 [5] can structure an authorization, but importing an encoding library is not signature verification. ERC-7730 [6] describes clear-signing data; a host-side modal is not a Ledger secure screen. ERC-4337 [4] provides account-abstraction infrastructure; session-key restrictions require appropriate account modules and are future work here. None of these integrations is represented as deployed in the demonstrator.

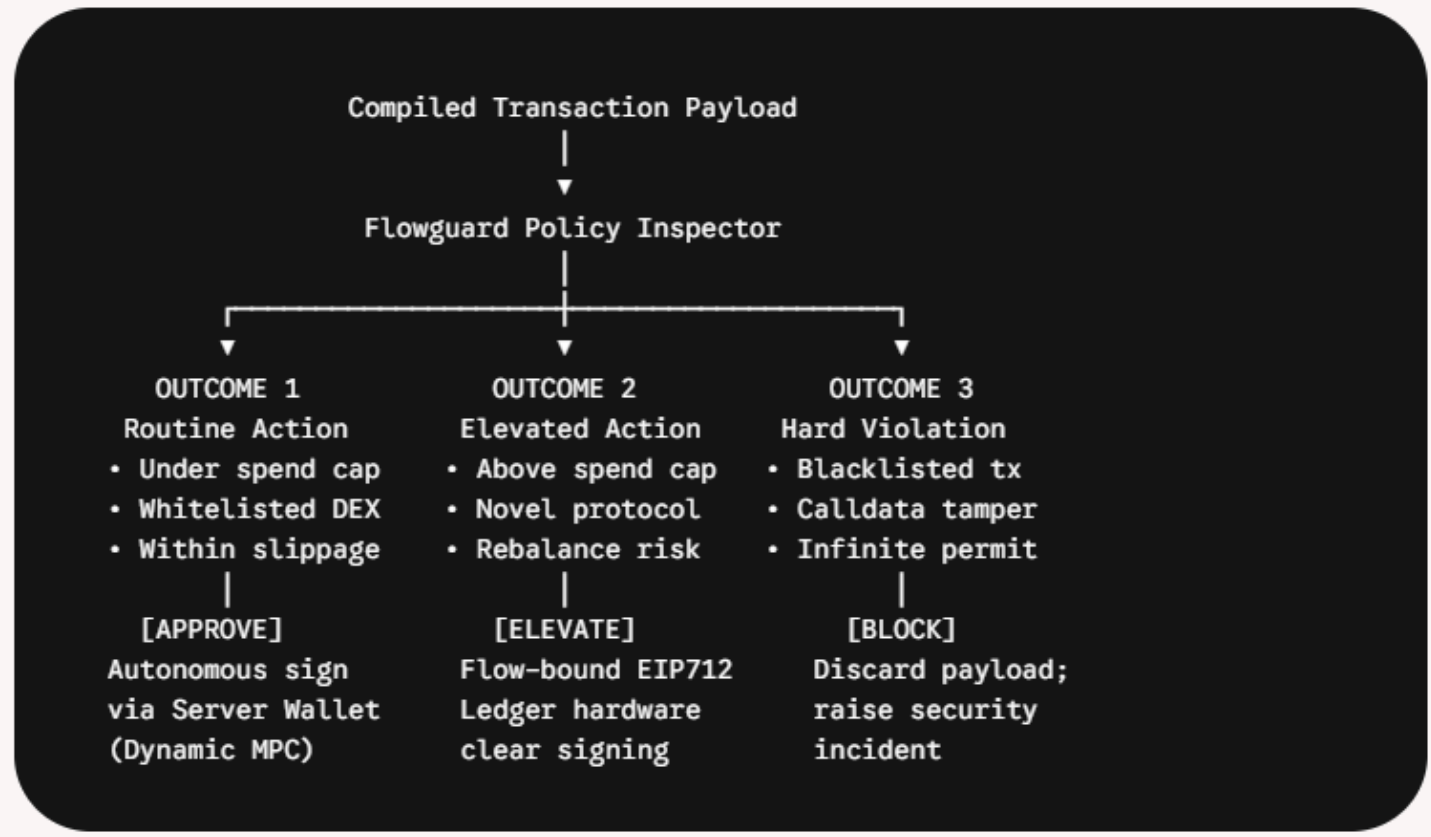


Figure 4 (conceptual / proposed): Tri-state policy outcomes. The demo implements the three outcomes as local decisions only. The three outcomes follow Flowguard [11]. Blacklist, calldata-tamper, infinite-permit and novel-protocol checks, and all signing paths, are not implemented.

5. Implemented Policy and Intent Format

The implementation is index.html in this delivery. Supported actions are SWAP_EXACT_IN and TREASURY_REBALANCE. Both require a valid positive expected output and minimum output. Other actions are rejected. The illustrative router address is a configuration fixture, not a deployment verified by this work.

- **Hourly budget:** Human-approved spend also counts toward the rolling hourly total. The initial 0.002 ETH demonstration fixture ages out after one hour.
- **Autonomous Path:** Amount must be at most 0.005 ETH and total simulated spending during the preceding hour, including the candidate, must be at most 0.02 ETH. Otherwise the action requests simulated human review.
- **Gas Bounds:** Gas must be present, finite, nonnegative, and at most 15 Gwei.
- **Allowlists:** The target must equal the configured router fixture and the recipient must equal the configured vault fixture. The action must be `SWAP_EXACT_IN` or `TREASURY_REBALANCE`. Anything else is blocked.
- **Slippage Constraint:** The ratio $\frac{EstReturn - MinReturn}{EstReturn}$ must be no greater than 1%. This ratio measures the supplied quote tolerance, not price impact or fair value. Quotes are local inputs; no authenticated market price is obtained.

For deterministic demonstration accounting, amounts support at most nine decimal places of ETH, and the rolling budget sums integer nanounits. This is a simulation precision limit, not Ethereum's native unit specification. A deployed implementation must use integer token base units and verified token decimals, including six decimals for the fixed USDC example and eighteen for WETH where those deployments match the configured assets.


```
{
  "schemaVersion": 1,
  "policyVersion": "simulation-v1",
  "intentId": "tis-19a2b3c4d5e-k7x2",
  "action": "SWAP_EXACT_IN",
  "protocol": "UNISWAP_V3",
  "chainId": 84532,
  "proposer": "AgentPlanner_ReAct",
  "targetContract": "0x94cC0AaC535CCDB3C01d6787d6413C739ae12bc4",
```

```
  "amountEth": 0.002,

  "estReturn": 6.12,

  "minReturn": 6.08,

  "recipient": "0x1111111111111111111111111111111111111111",

  "nonce": 105,

  "deadline": 1793232300,

  "contextExcerpt": "Cyclic spread observed (0.65%). Proposing swap of 0.002 ETH on
Base Sepolia Unis"

}
```


The intent includes `schemaVersion`, `policyVersion`, `intentId`, `action`, `protocol`, `chainId`, `proposer`, `targetContract`, `amountEth`, `estReturn`, `minReturn`, `recipient`, `nonce`, `deadline`, and `contextExcerpt` (the first 80 characters of the prompt text, kept for display only and never interpreted as authorization). The target must be the configured router fixture, and the recipient must be the configured vault fixture, both compared case-insensitively. Neither address is a verified deployment. Amounts and quotes are JSON numbers in this simulation. A deployed system must use integer base units. Sorted-key JSON serialization is hashed with SHA-256. This is an intent digest, not a transaction hash, EIP-712 signature, calldata hash, or signed PDR. If hashing fails on an otherwise valid intent, the run is blocked; there is no weaker fallback.

6. Approval Lifecycle and Implementation Limits

A fresh run invalidates an older pending request. The selected intent and its digest are retained for simulated human review. Confirmation checks current gas, halt state, expiry, nonce, domain/version, target, quotes, and amount policy; it recomputes the digest and rejects a mismatch. It rechecks after asynchronous hashing before changing local spending state.

A pending confirmation is consumed once, including failure. Rejection, a new scenario, and a new run invalidate pending review. Completed simulated approvals consume their nonce. Emergency halt prevents further simulated dispatch. Exceptions clear the running state and re-enable the interface. These are local state-machine properties, not a substitute for cryptographic authorization.

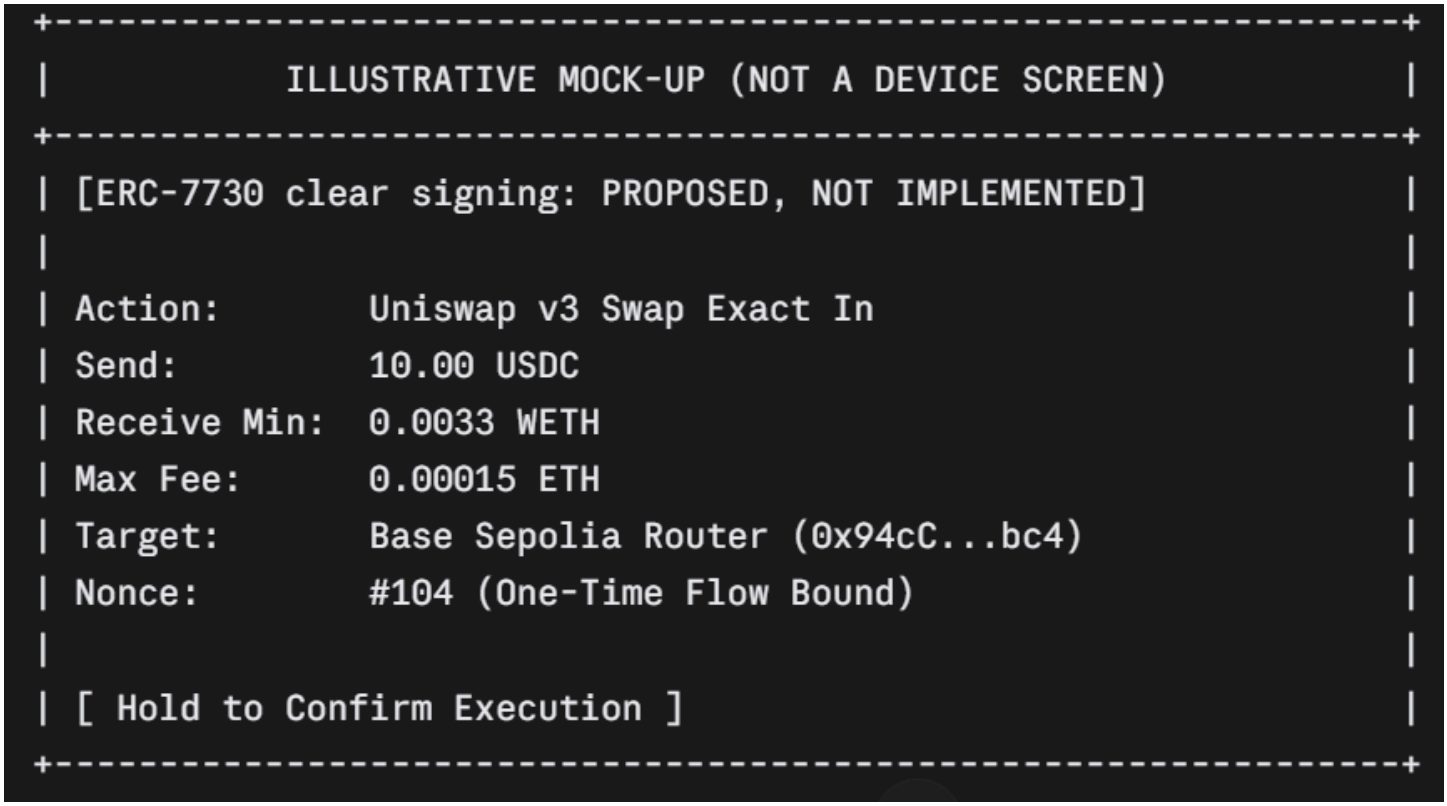


Figure 5 (conceptual / proposed): Illustrative ERC-7730 clear-signing display. No Ledger device or ERC-7730 descriptor is used. The demo shows a host-side modal labeled as a simulation, and its fields and values differ from this mock-up.

The rolling hour is maintained in memory using timestamped spend events. Reloading the page resets budgets, consumed nonces, and audit history. The initial 0.002 ETH event is a demonstration fixture. Audit events and displayed records are local and not tamper-evident. No custody or production accounting guarantee is claimed.

The deliverable is the browser simulation only; no backend is included. A future backend should be delivered as source with pinned dependencies, authenticated signer integration, protocol-specific tests, and reproducible execution artifacts.

7. Validation and Results

The accompanying `test_demo.py` uses Playwright with Chromium against the delivered HTML. External asset requests are blocked during functional checks to isolate local policy behavior. The results file records **56 passing checks and zero failures** in that run. The four preset expected outcomes are: routine APPROVED, memory-injection preset BLOCKED, excessive slippage BLOCKED, and larger rebalance ELEVATED. A separate check isolates the target allowlist by changing only the target.

Table 1: Preset scenarios and the checks that exercise them

Scenario / Preset	Rule(s) exercised	Observed result
A. Routine Arbitrage Swap	Spend ≤ 0.005 ETH; hourly total ≤ 0.02 ETH; slippage ≤ 1%; router and vault allowlisted	PASS: APPROVED (Outcome 1)
B. Memory-Injection Preset	Five rules fail together: amount above the 0.10 ETH ceiling, unsupported action, non-router target, non-vault recipient, zero quote. All five are listed in the decision record.	PASS: BLOCKED (Outcome 3), no spend (check 2)
B′. Unknown target only	Valid amount, quote, action and recipient; only the target differs from the router. Exactly one violation is reported. This isolates the target allowlist.	PASS: BLOCKED (Outcome 3), no spend (check 27)
C. MEV / Slippage Breach	Quote tolerance 4.82% exceeds the 1% bound; exactly one violation	PASS: BLOCKED (Outcome 3)
D. High-Value Rebalance	Spend above 0.005 ETH and within the 0.10 ETH ceiling	PASS: ELEVATED (Outcome 2)

Check numbers refer to `id` values in `regression-results.json`.

The 56 regression checks cover invalid, blank and high gas; invalid amounts and quotes; the hardware ceiling; isolated target, recipient, action and address rules; boundary values (exactly 0.005 ETH, 0.10 ETH, 15 Gwei, 1% slippage, the 0.02 ETH hourly total, and the one-hour expiry edge);

confirmation after rejection; halt before dispatch, during a run, and during review; stale requests; altered intent digest and chain ID; expiry and consumed nonces; duplicate confirmation; hashing failure at dispatch and at confirmation; HTML injection through the target and prompt fields; an independent SHA-256 comparison; and four click-through flows that exercise the visible buttons, pills and badges. Zero gas is accepted as a valid simulation input. Every check fails the run on any uncaught page error. These tests can fail and directly exercise the delivered logic.

These outcomes establish only that the selected local regression cases behaved as expected. They do not establish 0% attack success across a population, adversarial robustness of an LLM, or live deployment readiness. No latency benchmark, benign-task utility rate, confidence interval for ASR, profitability result, or economic-loss comparison is reported. Per-check evidence is in `regression-results.json`.

8. Economic Strategies and Residual Risk

Concentrated-liquidity rebalancing and cyclic arbitrage remain motivating use cases, not implemented trading strategies. Narrow liquidity ranges can improve capital efficiency under specified assumptions but do not guarantee yield; they introduce rebalancing, liquidity, and loss risks. The capital-efficiency gain from concentrated liquidity depends on the chosen price range, volatility and fees, and is not quantified in this project.

A production cyclic strategy must compare receipts and all costs in the same numeraire. Include pool fees, priority fees, applicable L1 data fees on Base, slippage, and financing costs. A correctly designed single EVM transaction can revert all of its state changes, but a bundle of independent transactions is not automatically atomic, and a revert still incurs fees.

`eth_call` returns call output or an error, not a mined receipt status or automatic balance diffs. Checking a minimum output is not a gas-inclusive portfolio-solvency calculation. Simulation is state-dependent and can become stale. Slippage bounds and gas ceilings do not eliminate MEV; submission-path and sequencer behavior require chain-specific analysis.

9. Future Work and Acceptance Criteria

The next stage is a minimal reproducible backend, not more unverified integrations. Publish the source, schema, locked dependencies, selected chain/fork block, ABI provenance, configuration, and threat assumptions. Verify actual sender context, token units, native-asset handling, and decoded output shape. Reject unknown schema fields and invalid values according to a versioned contract.

Implement a signed PDR or equivalent authorization capability and independently configured signer. Verify signature, issuer, audience, chain/account domain, expiry, full execution binding, and atomic durable nonce consumption. Use persistent transaction accounting, reservations for concurrent requests, idempotency, receipt reconciliation, recovery, and key/authorization revocation.

For empirical model-security claims, specify model/version, system prompt, attack-corpus version and case IDs, repeated-run procedure, baselines, and separate benign and adversarial denominators. Report raw counts and uncertainty. For real-chain claims, provide authentic hashes, receipts, and before/after balances without exposing keys. For hardware claims, document the exact device/application/descriptor and verified authorization flow. Until then, keep those capabilities proposed.

10. Conclusion

The capstone demonstrates a local policy simulation with explicit scope, inspectable decisions, intent hashing, and tested approval lifecycle checks. It illustrates why planners should propose actions while separate components control authority. The results support the implemented local behaviors; they do not support universal security or deployment guarantees. Actions that stay inside the configured limits, including repeated small spends and unbounded gas cost, remain possible and are listed as residual risk in Section 3. Clear evidence boundaries make the project reproducible and identify the remaining engineering work.

References

- **[1] Patlan, A. S., Sheng, P., Hebbar, S. A., Mittal, P., and Viswanath, P.** *Real AI Agents with Fake Memories: Fatal Context Manipulation Attacks on Web3 Agents.* arXiv:2503.16248v3, 2025. Sections 5–6 and Figure 10. https://arxiv.org/abs/2503.16248
- **[2] Alqithami, S.** *Autonomous Agents on Blockchains: Standards, Execution Models, and Trust Boundaries.* arXiv:2601.04583v1, 2026. Section 8, Fig. 5, and evaluation guidance. https://arxiv.org/abs/2601.04583
- **[3] Zhang, M. (Aaron).** *AI_Finance teaching repository*, California Science and Technology University. https://github.com/aaron-recompile/AI_Finance/
- **[4] Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Gazso, K., and Hess, T.** *ERC-4337: Account Abstraction Using Alt Mempool.* Created 2021; living specification. https://eips.ethereum.org/EIPS/eip-4337
- **[5] EIP-712:** *Typed structured data hashing and signing.* Living specification. https://eips.ethereum.org/EIPS/eip-712
- **[6] ERC-7730:** *Structured Data for Clear Signing.* Living specification. https://eips.ethereum.org/EIPS/eip-7730
- **[7]** ElizaOS project. elizaOS: open-source framework for autonomous AI agents. https://github.com/elizaOS/eliza (accessed October 30, 2026)
- **[8]** Dynamic. *Server wallets overview*. https://www.dynamic.xyz/docs/wallets/server-wallets/overview (accessed October 30, 2026)
- **[9]** Ledger. *Clear signing.* https://developers.ledger.com/docs/clear-signing/ (accessed October 30, 2026)

- **[10]** LI.FI. *LI.FI documentation: introduction.* https://docs.li.fi/introduction (accessed October 30, 2026)
- **[11]** Flowguard (CircuitBreaker). *ETHGlobal New York 2026 showcase page* (Ledger, “AI Agents x Ledger” track, 5th place). https://ethglobal.com/showcase/flowguard-ruzp0. Source code: https://github.com/shivanshxyz/circuitbreaker. Live demo: https://circuitbreaker-eta.vercel.app (accessed October 30, 2026)

Appendix: Reproducing the Local Checks

Run these from the folder that contains `index.html` and `test_demo.py`. Install once:

```
python -m pip install playwright==1.63.0
python -m playwright install chromium
```

Start the local server (use `python3` if `python` is not found):

```
python -m http.server 8004 --bind 127.0.0.1
```

In a second terminal, run:

```
python test_demo.py
```

The script prints one PASS or FAIL line per check and writes `regression-results.json` (56 checks, 56 passed, 0 failed in the recorded run). The results file records the SHA-256 of the exact page tested: `5fd8d3a4825ede0d3c8853c4643d5036c8cf17ee318685ab22352396f208126b`. Any edit to `index.html`, even re-saving it with different line endings, changes the hash. Submit the file unmodified, or re-run the suite and update this value. Set `CAPSTONE_TEST_URL` to use a different server address. Functional checks block external requests, so Tailwind and icons are not loaded during the tests. The page’s visual styling needs internet access to those CDNs.